

Projeto de Banco de Dados - SISTEMA DE
GESTÃO ESCOLAR (SisGESC)

Escola de Idiomas

São Paulo
2026

Introdução

A gestão educacional atual enfrenta desafios cada vez mais complexos diante do crescimento das demandas administrativas, acadêmicas e financeiras nas instituições de ensino. Esse cenário se intensifica em escolas privadas de idiomas, que lidam com turmas de horários variados e a necessidade de acompanhamento contínuo do desempenho dos alunos e clareza com pais e responsáveis.

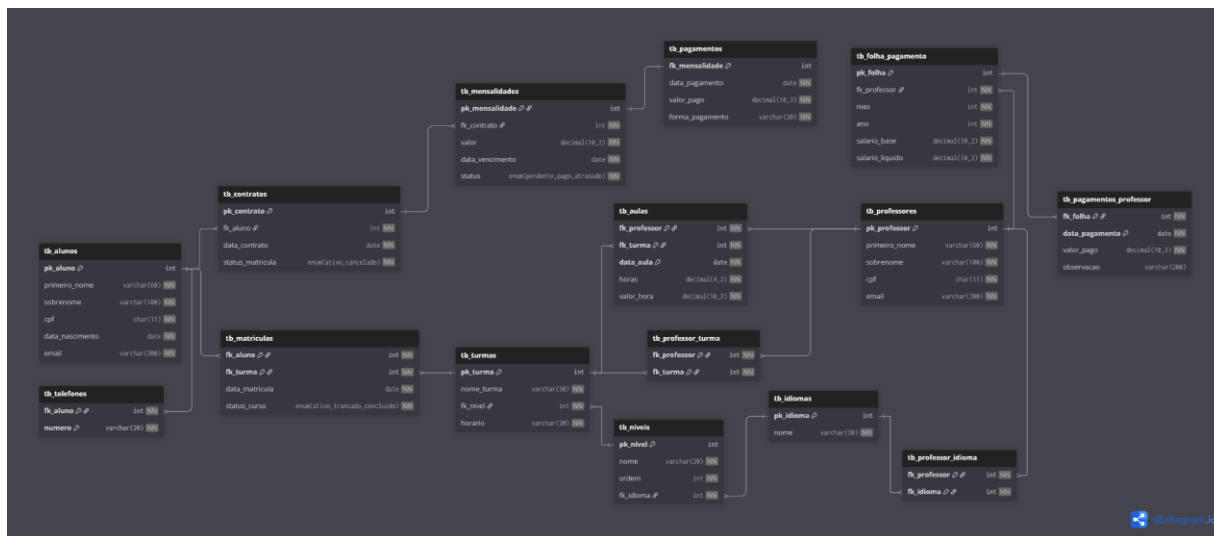
Pensando nessa situação, a gestão envolve o controle integrado de alunos, turmas, matrículas, professores e contratos financeiros. Mas ainda muitas instituições operam com processos manuais ou sistemas fragmentados, o que compromete a eficiência operacional e a qualidade das informações. Dessa forma, surgem problemas recorrentes, que estão na dificuldade do acompanhamento da evasão escolar e erros de faturamento por conta da falta de integração entre o setor financeiro e a secretaria acadêmica e além da falta de transparência nas informações disponibilizadas para alunos, pais e responsáveis.

Vendo essas limitações, a adoção de sistemas informatizados integrados, como os ERP (Enterprise Resource Planning), apresenta-se como uma solução estratégica para centralizar, padronizar e otimizar a gestão institucional. Nesse sentido, propõe-se o desenvolvimento do SisGESC, voltado à integração dos processos acadêmicos, de recursos humanos e financeiros de uma escola de idiomas.

Esta modelagem do sistema foi elaborada considerando as particularidades desse tipo de instituição, refletidas na estrutura do banco de dados. Destacam-se o relacionamento entre alunos e turmas por meio de matrículas, a organização das turmas por idioma e nível de proficiência, a alocação de professores e o controle financeiro baseado em contratos, mensalidades e pagamentos.

Por fim como perspectiva de evolução, propõe-se a futura integração de soluções de Business Intelligence (BI) e Inteligência Artificial no objetivo de melhorar a análise de dados, apoiar a tomada de decisões estratégicas e aprimorar o acompanhamento do desempenho acadêmico e financeiro da instituição.

DER



MODELAGEM ESTRELA

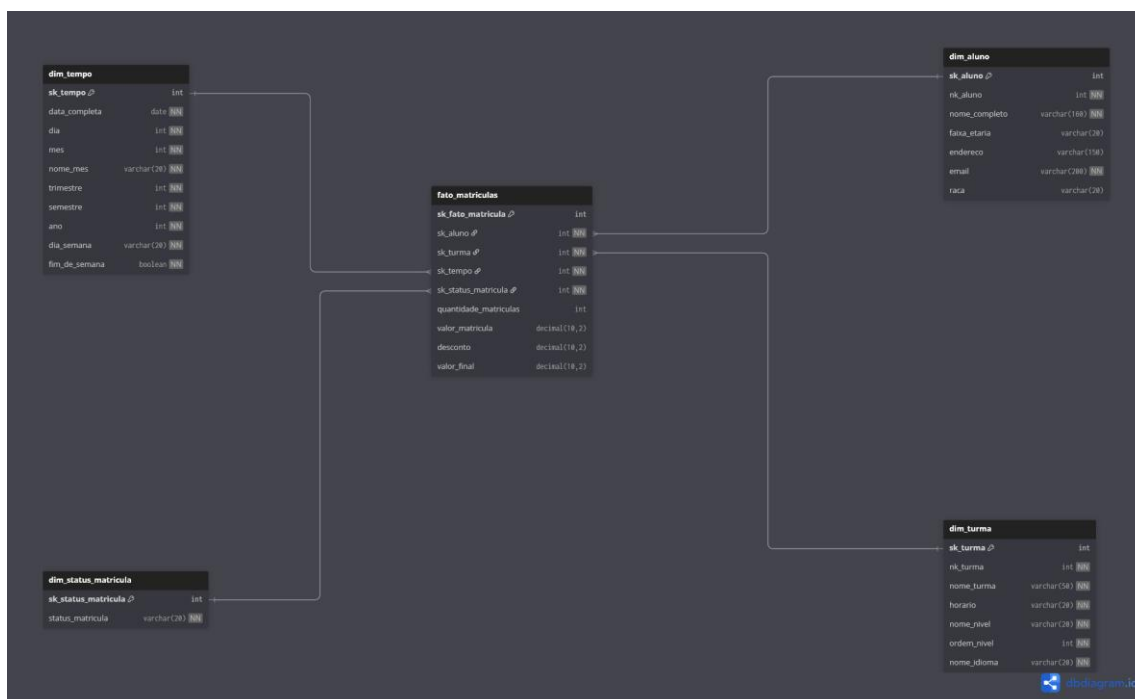


Diagrama de dados feito com a ferramenta dbdiagram.io

Link do Der disponivel em:

<https://dbdiagram.io/d/DER-SisGesc-69d556f980896296843e5c3d>

<https://dbdiagram.io/d/Esquema-Estrela-69ffdf57a923b9472684201>

Dicionário de Dados

Esta seção descreve detalhadamente todas as tabelas, atributos, tipos de dados e restrições do banco de dados modelado no DER apresentado.

TABELA: tb_alunos

Descrição:

Armazena os dados cadastrais dos alunos da instituição. Esta tabela é fundamental para o sistema, pois serve como base para relacionamentos com matrícula, contratos e controle financeiro.

Nome	Tipo de Dado	Restrição	Descrição	Observação
pk_aluno	INT	PK, NOT NULL	Identificador único do aluno no sistema	Auto incremento
primeiro_nome	VARCHAR(60)	NOT NULL	Primeiro nome do aluno	
sobrenome	VARCHAR(100)	NOT NULL	Sobrenome do aluno	
cpf	CHAR(11)	NOT NULL, UNIQUE	Cadastro de Pessoa Física do aluno	Utilizado para evitar duplicidade
data_nascimento	DATE	NOT NULL	Data de nascimento do aluno	
email	VARCHAR(200)	NOT NULL, UNIQUE	Endereço de e-mail do aluno	Comunicação institucional

TABELA: tb_telefones

Descrição:

Armazena os telefones dos alunos, permitindo múltiplos contatos por aluno. A chave primária é composta por fk_aluno e numero, impedindo o cadastro do mesmo número para o mesmo aluno.

Nome	Tipo de Dado	Restrição	Descrição	Observação
fk_aluno	INT	PK, FK, NOT NULL	Referência ao aluno dono do telefone	Relaciona com tb_alunos. ON DELETE CASCADE
numero	VARCHAR(20)	PK, NOT NULL	Número de telefone do aluno	Compõe a chave primária junto com fk_aluno

TABELA: tb_idiomas

Descrição:

Catálogo dos idiomas oferecidos pela escola, permitindo a categorização das turmas e professores.

Nome	Tipo de Dado	Restrição	Descrição	Observação
pk_idioma	INT	PK, NOT NULL	Identificador único do idioma	Auto incremento
nome	VARCHAR(20)	NOT NULL, UNIQUE	Nome do idioma	Ex: Inglês, Espanhol, Francês

TABELA: tb_niveis

Descrição:

Define os níveis de aprendizado dos cursos, organizando a progressão acadêmica dos alunos dentro de cada idioma.

Nome	Tipo de Dado	Restrição	Descrição	Observação
pk_nivel	INT	PK, NOT NULL	Identificador único do nível	Auto incremento
nome	VARCHAR(20)	NOT NULL	Nome do nível	Ex: Básico, Intermediário, Avançado
ordem	INT	NOT NULL	Ordem hierárquica do nível na progressão	UNIQUE junto com fk_idioma
fk_idioma	INT	FK, NOT NULL	Referência ao idioma ao qual o nível pertence	Relaciona com tb_idiomas. ON DELETE RESTRICT

TABELA: tb_turmas

Descrição:

Representa as turmas disponíveis na instituição, vinculando idioma, nível e horário. Cada turma é associada a um nível de aprendizado.

Nome	Tipo de Dado	Restrição	Descrição	Observação
pk_turma	INT	PK, NOT NULL	Identificador único da turma	Auto incremento
nome_turma	VARCHAR(50)	NOT NULL	Nome ou código de identificação da turma	Ex: Inglês Básico A
fk_nivel	INT	FK, NOT NULL	Referência ao nível da turma	Relaciona com tb_niveis. ON DELETE RESTRICT
horario	VARCHAR(20)	NOT NULL	Horário de realização das aulas	Ex: 19:00 - 21:00

TABELA: tb_matriculas

Descrição:

Controla o vínculo entre alunos e turmas, registrando a participação do aluno em determinado curso. A chave primária composta (fk_aluno, fk_turma) impede matrícula duplicada na mesma turma.

Nome	Tipo de Dado	Restrição	Descrição	Observação
fk_aluno	INT	PK, FK, NOT NULL	Referência ao aluno matriculado	Relaciona com tb_alunos. ON DELETE CASCADE
fk_turma	INT	PK, FK, NOT NULL	Referência à turma escolhida	Relaciona com tb_turmas. ON DELETE RESTRICT
data_matricula	DATE	NOT NULL	Data em que a matrícula foi realizada	
status_curso	ENUM	NOT NULL	Situação do aluno no curso	Valores: 'ativo', 'trancado', 'concluido'. DEFAULT: 'ativo'

TABELA: tb_contratos

Descrição:

Representa o vínculo financeiro entre o aluno e a instituição, registrando a formalização da matrícula sob o ponto de vista financeiro.

Nome	Tipo de Dado	Restrição	Descrição	Observação
pk_contrato	INT	PK, NOT NULL	Identificador único do contrato	Auto incremento
fk_aluno	INT	FK, NOT NULL	Referência ao aluno titular do contrato	Relaciona com tb_alunos. ON DELETE RESTRICT
data_contrato	DATE	NOT NULL	Data de formalização do contrato	
status_matricula	ENUM	NOT NULL	Situação do contrato	Valores: 'ativo', 'cancelado'. DEFAULT: 'ativo'

TABELA: tb_mensalidades

Descrição:

Controla as cobranças periódicas associadas aos contratos dos alunos. O UNIQUE em (fk_contrato, data_vencimento) impede cobranças duplicadas no mesmo mês para o mesmo contrato.

Nome	Tipo de Dado	Restrição	Descrição	Observação
pk_mensalidade	INT	PK, NOT NULL	Identificador único da mensalidade	Auto incremento
fk_contrato	INT	FK, NOT NULL	Referência ao contrato do aluno	Relaciona com tb_contratos. ON DELETE CASCADE
valor	DECIMAL(10,2)	NOT NULL	Valor nominal da mensalidade	
data_vencimento	DATE	NOT NULL	Data limite para pagamento	UNIQUE junto com fk_contrato
status	ENUM	NOT NULL	Situação da cobrança	Valores: 'pendente', 'pago', 'atrasado'. DEFAULT: 'pendente'

TABELA: tb_pagamentos

Descrição:

Registra os pagamentos efetuados pelos alunos, permitindo rastrear quitação de mensalidades e métodos de pagamento utilizados.

Nome	Tipo de Dado	Restrição	Descrição	Observação
fk_mensalidade	INT	PK, FK, NOT NULL	Referência à mensalidade paga	Relaciona com tb_mensalidades. ON DELETE RESTRICT
data_pagamento	DATE	NOT NULL	Data em que o pagamento foi realizado	
valor_pago	DECIMAL(10,2)	NOT NULL	Valor efetivamente pago pelo aluno	CHECK: valor maior que zero
forma_pagamento	VARCHAR(20)	NOT NULL	Método de pagamento utilizado	Ex: PIX, Cartão, Boleto

TABELA: tb_professores

Descrição:

Armazena os dados dos professores responsáveis pelas turmas. Esta tabela é essencial para o controle acadêmico e de recursos humanos.

Nome	Tipo de Dado	Restrição	Descrição	Observação
pk_professor	INT	PK, NOT NULL	Identificador único do professor	Auto incremento
primeiro_nome	VARCHAR(60)	NOT NULL	Primeiro nome do professor	
sobrenome	VARCHAR(100)	NOT NULL	Sobrenome do professor	
cpf	CHAR(11)	NOT NULL, UNIQUE	CPF do professor	Sem formatação, evita duplicidade
email	VARCHAR(200)	NOT NULL, UNIQUE	E-mail do professor	

TABELA: tb_professor_turma

Descrição:

Tabela associativa que implementa o relacionamento muitos-para-muitos entre professores e turmas, permitindo que um professor leccione em várias turmas e vice-versa.

Nome	Tipo de Dado	Restrição	Descrição	Observação
fk_professor	INT	PK, FK, NOT NULL	Referência ao professor	Relaciona com tb_professores. ON DELETE CASCADE
fk_turma	INT	PK, FK, NOT NULL	Referência à turma	Relaciona com tb_turmas. ON DELETE RESTRICT

TABELA: tb_professor_idioma

Descrição:

Relaciona professores aos idiomas que estão habilitados a lecionar. Implementa um relacionamento muitos-para-muitos entre professores e idiomas.

Nome	Tipo de Dado	Restrição	Descrição	Observação
fk_professor	INT	PK, FK, NOT NULL	Referência ao professor	Relaciona com tb_professores.

				ON DELETE CASCADE
fk_idioma	INT	PK, FK, NOT NULL	Referência ao idioma	Relaciona com tb_idiomas. ON DELETE RESTRICT

TABELA: tb_folha_pagamento

Descrição:

Controla a folha de pagamento mensal dos professores. O UNIQUE em (fk_professor, mes, ano) impede a geração de folhas duplicadas para o mesmo professor no mesmo período.

Nome	Tipo de Dado	Restrição	Descrição	Observação
pk_folha	INT	PK, NOT NULL	Identificador único da folha de pagamento	Auto incremento
fk_professor	INT	FK, NOT NULL	Referência ao professor	Relaciona com tb_professores. ON DELETE RESTRICT
mes	INT	NOT NULL	Mês de referência da folha (1 a 12)	UNIQUE com fk_professor e ano
ano	INT	NOT NULL	Ano de referência da folha	UNIQUE com fk_professor e mes
salario_base	DECIMAL(10,2)	NOT NULL	Salário bruto do professor no período	CHECK: valor maior que zero
salario_liquido	DECIMAL(10,2)	NOT NULL	Salário após descontos	CHECK: valor maior que zero

TABELA: tb_aulas

Descrição:

Registra cada aula ministrada por um professor em uma turma. A chave primária composta (fk_professor, fk_turma, data_aula) impede duplicidade de registro para o mesmo professor, turma e data.

Nome	Tipo de Dado	Restrição	Descrição	Observação
fk_professor	INT	PK, FK, NOT NULL	Referência ao professor que ministrou a aula	Relaciona com tb_professores. ON DELETE CASCADE
fk_turma	INT	PK, FK, NOT NULL	Referência à turma em que a aula ocorreu	Relaciona com tb_turmas. ON DELETE RESTRICT
data_aula	DATE	PK, NOT NULL	Data em que a aula foi ministrada	Compõe a chave primária composta
horas	DECIMAL(4,2)	NOT NULL	Duração da aula em horas	CHECK: valor maior que zero
valor_hora	DECIMAL(10,2)	NOT NULL	Valor por hora trabalhada pelo professor	CHECK: valor maior que zero

TABELA: tb_pagamentos_professor

Descrição:

Registra os pagamentos realizados aos professores, vinculados à folha de pagamento. A chave primária composta (fk_folha, data_pagamento) permite múltiplos pagamentos parciais em datas diferentes para a mesma folha.

Nome	Tipo de Dado	Restrição	Descrição	Observação
fk_folha	INT	PK, FK, NOT NULL	Referência à folha de pagamento	Relaciona com tb_folha_pagamento. ON DELETE CASCADE

data_pagamento	DATE	PK, NOT NULL	Data em que o pagamento foi efetuado	Compõe a chave primária composta
valor_pago	DECIMAL(10,2)	NOT NULL	Valor transferido ao professor	CHECK: valor maior que zero
observacao	VARCHAR(200)	NULL	Anotações opcionais sobre o pagamento	Campo opcional

Regras de Negócio

Regra 1 – CPF único por aluno

Cada aluno deve possuir um CPF exclusivo no sistema, não sendo permitido o cadastro de dois alunos com o mesmo CPF.

Implementação: restrição UNIQUE no campo cpf da tabela tb_alunos.

Regra 2 – Associação obrigatória entre nível e idioma

Todo nível de ensino deve estar obrigatoriamente vinculado a um idioma previamente cadastrado no sistema, garantindo a organização hierárquica dos cursos e evitar níveis sem contexto acadêmico.

Implementação: chave estrangeira fk_idioma na tabela tb_niveis, referenciando tb_idiomas(pk_idioma).

Regra 3 – Validação de aluno e turma na matrícula

Uma matrícula só poderá ser realizada quando o aluno e a turma estiverem previamente cadastrados no sistema. impedindo registros órfãos e preservar a integridade relacional.

Implementação: chaves estrangeiras fk_aluno e fk_turma na tabela tb_matriculas.

Regra 4 – Possibilidade de múltiplos pagamentos para a mesma folha salarial

O sistema dpemite a realização de mais de um pagamento vinculado à mesma folha salarial de um professor, possibilitando correções ou complementações em casos de erro no valor inicialmente depositado.

Implementação: tabela tb_pagamentos_professor vinculada à tabela tb_folha_pagamento por meio da chave estrangeira fk_folha, utilizando chave primária composta (fk_folha, data_pagamento).

Regra 5 – Relacionamento entre professores e turmas

O sistema deve permitir que um professor lecione em várias turmas e que uma turma possua mais de um professor.

Implementação: tabela associativa tb_professor_turma com chave primária composta (fk_professor, fk_turma).

Regra 6 – Pagamento Único de Mensalidade

O sistema não permite pagamentos parciais de mensalidades.

Implementação: Cada mensalidade deve ser quitada integralmente em um único pagamento, garantindo que exista apenas um registro de pagamento para cada mensalidade cadastrada no sistema.

/*SCRIPT SQL*/

```
CREATE DATABASE escola_idiomas;
USE escola_idiomas;
CREATE TABLE tb_alunos(pk_aluno INT AUTO_INCREMENT PRIMARY KEY,
primeiro_nome VARCHAR(60) NOT NULL,
sobrenome VARCHAR(100) NOT NULL,
cpf CHAR(11) NOT NULL UNIQUE,
data_nascimento DATE NOT NULL,
email VARCHAR(200) NOT NULL UNIQUE);

CREATE TABLE tb_telefones(fk_aluno INT NOT NULL,
numero VARCHAR(20) NOT NULL,
PRIMARY KEY (fk_aluno, numero),
FOREIGN KEY (fk_aluno) REFERENCES tb_alunos(pk_aluno) ON DELETE CASCADE);

CREATE TABLE tb_idiomas(
pk_idioma INT AUTO_INCREMENT PRIMARY KEY,
nome VARCHAR(20) NOT NULL UNIQUE);

CREATE TABLE tb_niveis(pk_nivel INT AUTO_INCREMENT PRIMARY KEY,
nome VARCHAR(20) NOT NULL,
ordem INT NOT NULL,
fk_idioma INT NOT NULL,
FOREIGN KEY (fk_idioma) REFERENCES tb_idiomas(pk_idioma) ON DELETE RESTRICT,
UNIQUE (fk_idioma, ordem));

CREATE TABLE tb_turmas(pk_turma INT AUTO_INCREMENT PRIMARY KEY,
nome_turma VARCHAR(50) NOT NULL,
fk_nivel INT NOT NULL,
horario VARCHAR(20) NOT NULL,
FOREIGN KEY (fk_nivel) REFERENCES tb_niveis(pk_nivel) ON DELETE RESTRICT );

CREATE TABLE tb_matriculas(fk_aluno INT NOT NULL,
fk_turma INT NOT NULL,
data_matricula DATE NOT NULL,
status_curso ENUM('ativo', 'trancado', 'concluido') NOT NULL DEFAULT 'ativo',
PRIMARY KEY (fk_aluno, fk_turma),
FOREIGN KEY (fk_aluno) REFERENCES tb_alunos(pk_aluno) ON DELETE CASCADE,
FOREIGN KEY (fk_turma) REFERENCES tb_turmas(pk_turma) ON DELETE RESTRICT);
CREATE TABLE tb_contratos(pk_contrato INT AUTO_INCREMENT PRIMARY KEY, fk_aluno INT
NOT NULL, data_contrato DATE NOT NULL,
status_matricula ENUM('ativo', 'cancelado') NOT NULL DEFAULT 'ativo',
FOREIGN KEY (fk_aluno) REFERENCES tb_alunos(pk_aluno) ON DELETE RESTRICT);
CREATE TABLE tb_mensalidades(pk_mensalidade INT AUTO_INCREMENT PRIMARY KEY,
fk_contrato INT NOT NULL,
valor DECIMAL(10, 2) NOT NULL CHECK (valor > 0),
data_vencimento DATE NOT NULL,
status ENUM('pendente', 'pago', 'atrasado') NOT NULL DEFAULT 'pendente',
UNIQUE (fk_contrato, data_vencimento),
```

```
FOREIGN KEY (fk_contrato) REFERENCES tb_contratos(pk_contrato) ON DELETE CASCADE);
```

```
CREATE TABLE tb_pagamentos(fk_mensalidade INT PRIMARY KEY,  
data_pagamento DATE NOT NULL,  
valor_pago DECIMAL(10, 2) NOT NULL CHECK (valor_pago > 0),  
forma_pagamento VARCHAR(20) NOT NULL,  
FOREIGN KEY (fk_mensalidade) REFERENCES tb_mensalidades(pk_mensalidade) ON DELETE  
RESTRICT);
```

```
CREATE TABLE tb_professores(pk_professor INT AUTO_INCREMENT PRIMARY KEY,  
primeiro_nome VARCHAR(60) NOT NULL,  
sobrenome VARCHAR(100) NOT NULL,  
cpf CHAR(11) NOT NULL UNIQUE,  
email VARCHAR(200) NOT NULL UNIQUE);
```

```
CREATE TABLE tb_professor_turma(fk_professor INT NOT NULL,  
fk_turma INT NOT NULL,  
PRIMARY KEY (fk_professor, fk_turma),  
FOREIGN KEY (fk_professor) REFERENCES tb_professores(pk_professor) ON DELETE  
CASCADE,  
FOREIGN KEY (fk_turma) REFERENCES tb_turmas(pk_turma) ON DELETE RESTRICT);
```

```
CREATE TABLE tb_professor_idioma(fk_professor INT NOT NULL,  
fk_idioma INT NOT NULL,  
PRIMARY KEY (fk_professor, fk_idioma),  
FOREIGN KEY (fk_professor) REFERENCES tb_professores(pk_professor) ON DELETE  
CASCADE,  
FOREIGN KEY (fk_idioma) REFERENCES tb_idiomas(pk_idioma) ON DELETE RESTRICT);
```

```
CREATE TABLE tb_folha_pagamento( pk_folha INT AUTO_INCREMENT PRIMARY KEY,  
fk_professor INT NOT NULL,  
mes INT NOT NULL,  
ano INT NOT NULL,  
salario_base DECIMAL(10, 2) NOT NULL CHECK (salario_base >= 0),  
salario_liquido DECIMAL(10, 2) NOT NULL (salario_liquido >= 0), UNIQUE  
(fk_professor, mes, ano),  
FOREIGN KEY (fk_professor) REFERENCES tb_professores(pk_professor) ON DELETE  
RESTRICT);
```

```
CREATE TABLE tb_aulas( fk_professor INT NOT NULL,  
fk_turma INT NOT NULL,  
data_aula DATE NOT NULL,  
horas DECIMAL(4, 2) NOT NULL NOT NULL CHECK (horas > 0),  
valor_hora DECIMAL(10, 2) NOT NULL CHECK (valor_hora > 0),  
PRIMARY KEY (fk_professor, fk_turma, data_aula), FOREIGN KEY (fk_professor)  
REFERENCES tb_professores(pk_professor) ON DELETE CASCADE, FOREIGN KEY (fk_turma)  
REFERENCES tb_turmas(pk_turma) ON DELETE RESTRICT);
```

```
CREATE TABLE tb_pagamentos_professor( fk_folha INT NOT NULL,  
data_pagamento DATE NOT NULL,  
valor_pago DECIMAL(10, 2) NOT NULL CHECK (valor_pago > 0),
```

```
observacao VARCHAR(200),  
PRIMARY KEY (fk_folha, data_pagamento), FOREIGN KEY (fk_folha) REFERENCES  
tb_folha_pagamento(pk_folha) ON DELETE CASCADE);
```

/*INSERTS*/

```
INSERT IGNORE INTO tb_idiomas (pk_idioma, nome) VALUES
(1, 'Inglês'), (2, 'Espanhol'), (3, 'Francês'), (4, 'Alemão'), (5, 'Italiano');
INSERT IGNORE INTO tb_niveis (pk_nivel, nome, ordem, fk_idioma) VALUES
(1, 'Iniciante A1', 1, 1), (2, 'Intermediário B1', 2, 1), (3, 'Avançado C1', 3,
1), (4, 'Básico 1', 1, 2), (5, 'Intermediário 1', 2, 2), (6, 'Iniciante A1', 1,
3), (7, 'Iniciante A1', 1, 4);
INSERT IGNORE INTO tb_professores (pk_professor, primeiro_nome, sobrenome, cpf,
email) VALUES
(1, 'Ricardo', 'Oliveira', '12345678901', 'ricardo@escola.com'), (2, 'Elena',
'Silva', '98765432100', 'elena@escola.com'), (3, 'Marcus', 'Weber', '45612378911',
'marcus@escola.com'), (4, 'Beatriz', 'Costa', '78945612322',
'beatriz@escola.com');
INSERT IGNORE INTO tb_professor_idioma (fk_professor, fk_idioma) VALUES /*associa
idiomas a professores*/
(1, 1), (1, 2), (2, 3), (3, 4), (4, 1);
INSERT IGNORE INTO tb_turmas (pk_turma, nome_turma, fk_nivel, horario) VALUES
(1, 'ENG-A1-SABADO', 1, '09:00 - 12:00'), (2, 'ESP-B1-TER-QUI', 4, '19:00 -
20:30'), (3, 'FRA-A1-SEG-QUA', 6, '18:00 - 19:30'), (4, 'ENG-B1-SEG-QUA', 2,
'20:30 - 22:00');
INSERT IGNORE INTO tb_alunos (pk_aluno, primeiro_nome, sobrenome, cpf,
data_nascimento, email) VALUES
(1, 'João', 'Ferreira', '11122233344', '2000-05-15', 'joao@email.com'), (2,
'Maria', 'Santos', '55566677788', '1995-10-20', 'maria@email.com'), (3, 'Pedro',
'Souza', '22233344455', '2002-03-12', 'pedro@email.com'), (4, 'Ana', 'Juliana',
'44455566699', '1988-12-05', 'ana@email.com'), (5, 'Lucas', 'Lima', '88899911122',
'1997-07-25', 'lucas@email.com');
INSERT IGNORE INTO tb_telefones (fk_aluno, numero) VALUES /*id 1 mostra caso um
aluno tenha mais de um telefone*/
(1, '11988887777'), (1, '11988887766'), (2, '11977776666'), (3, '21999998888'),
(4, '11955554444'), (5, '19966665555');
INSERT IGNORE INTO tb_matriculas (fk_aluno, fk_turma, data_matricula,
status_curso) VALUES
(1, 1, '2024-01-10', 'ativo'),
(2, 2, '2024-01-12', 'ativo'),
(3, 1, '2024-01-15', 'ativo'),
(4, 3, '2024-01-20', 'trancado'),
(5, 4, '2024-01-22', 'ativo');
INSERT IGNORE INTO tb_contratos (pk_contrato, fk_aluno, data_contrato,
status_matricula) VALUES
(1, 1, '2024-01-10', 'ativo'), (2, 2, '2024-01-12', 'ativo'), (3, 3, '2024-01-15',
'ativo'), (4, 4, '2024-01-20', 'cancelado'), /*Contrato cancelado refletindo a
matrícula trancada*/
(5, 5, '2024-01-22', 'ativo');
INSERT IGNORE INTO tb_mensalidades (pk_mensalidade, fk_contrato, valor,
data_vencimento, status) VALUES
(1, 1, 350.00, '2024-02-10', 'pago'), (2, 1, 350.00, '2024-03-10', 'pago'), (3, 1,
350.00, '2024-04-10', 'pendente'), (4, 2, 320.00, '2024-02-15', 'pago'), (5, 2,
320.00, '2024-03-15', 'atrasado'), /*simulação de status do pagamento*/ (6, 3,
```

```

350.00, '2024-02-15', 'pago'), (7, 3, 350.00, '2024-03-15', 'pago'), (8, 3,
350.00, '2024-04-15', 'pago'),
(9, 4, 380.00, '2024-02-20', 'pago'), (10, 5, 350.00, '2024-02-22', 'pago'), (11,
5, 350.00, '2024-03-22', 'pendente');
INSERT IGNORE INTO tb_pagamentos (fk_mensalidade, data_pagamento, valor_pago,
forma_pagamento) VALUES
(1, '2024-02-08', 350.00, 'Cartão de Crédito'), (2, '2024-03-10', 350.00, 'PIX'),
(4, '2024-02-14', 320.00, 'PIX'), /*relatorio de pagamentos dos alunos*/ (6,
'2024-02-15', 350.00, 'Dinheiro'),
(7, '2024-03-14', 350.00, 'Cartão de Débito'),
(8, '2024-04-15', 350.00, 'PIX'),
(9, '2024-02-18', 380.00, 'Cartão de Crédito'), (10, '2024-02-20', 350.00, 'PIX');
INSERT IGNORE INTO tb_professor_turma (fk_professor, fk_turma) VALUES
(1, 1),
(1, 2), /*professores podem lecionar mais de um idioma*/
(2, 3),
(4, 4);
INSERT IGNORE INTO tb_aulas (fk_professor, fk_turma, data_aula, horas, valor_hora)
VALUES
(1, 1, '2024-03-02', 3.00, 50.00), (1, 1, '2024-03-09', 3.00, 50.00), (1, 1,
'2024-03-16', 3.00, 50.00), (1, 1, '2024-03-23', 3.00, 50.00), /*Histórico de
aulas ministradas com 'valor_hora' grava o valor histórico para auditoria e
cálculo da folha*/
(1, 2, '2024-03-05', 1.50, 55.00), (1, 2, '2024-03-07', 1.50, 55.00), (1, 2,
'2024-03-12', 1.50, 55.00), (1, 2, '2024-03-14', 1.50, 55.00), (2, 3, '2024-03-
04', 1.50, 60.00), (2, 3, '2024-03-06', 1.50, 60.00);
INSERT IGNORE INTO tb_folha_pagamento (pk_folha, fk_professor, mes, ano,
salario_base, salario_liquido) VALUES
(1, 1, 3, 2024, 2500.00, 2350.00), (2, 2, 3, 2024, 1800.00, 1680.00);

INSERT IGNORE INTO tb_pagamentos_professor (fk_folha, data_pagamento, valor_pago,
observacao) VALUES
(1, '2024-04-05', 2350.00, 'Pagamento integral realizado via transferência
bancária'), (2, '2024-04-05', 1680.00, 'Pagamento integral realizado via PIX');
/*Permite pagamentos parciais, estornos e correções de valores incorretos lançados
na folha*/

```

/*Counts*/

```

SELECT 'tb_alunos' AS tabela, COUNT(*) AS total FROM tb_alunos
UNION ALL
SELECT 'tb_telefones', COUNT(*) FROM tb_telefones
UNION ALL
SELECT 'tb_idiomas', COUNT(*) FROM tb_idiomas
UNION ALL
SELECT 'tb_niveis', COUNT(*) FROM tb_niveis
UNION ALL
SELECT 'tb_turmas', COUNT(*) FROM tb_turmas
UNION ALL
SELECT 'tb_matriculas', COUNT(*) FROM tb_matriculas
UNION ALL

```

```
SELECT 'tb_contratos', COUNT(*) FROM tb_contratos
UNION ALL
SELECT 'tb_mensalidades', COUNT(*) FROM tb_mensalidades
UNION ALL
SELECT 'tb_pagamentos', COUNT(*) FROM tb_pagamentos
UNION ALL
SELECT 'tb_professores', COUNT(*) FROM tb_professores
UNION ALL
SELECT 'tb_professor_turma', COUNT(*) FROM tb_professor_turma
UNION ALL
SELECT 'tb_professor_idioma', COUNT(*) FROM tb_professor_idioma
UNION ALL
SELECT 'tb_folha_pagamento', COUNT(*) FROM tb_folha_pagamento
UNION ALL
SELECT 'tb aulas', COUNT(*) FROM tb_aulas
UNION ALL
SELECT 'tb_pagamentos_professor', COUNT(*) FROM tb_pagamentos_professor;
```

/*Selects basicos*/

```
SELECT a.pk_aluno,
       a.primeiro_nome,
       a.sobrenome,
       a.cpf,
       a.data_nascimento,
       a.email,
       t.numero AS telefone
FROM tb_alunos a
LEFT JOIN tb_telefones t
ON a.pk_aluno = t.fk_aluno
ORDER BY a.primeiro_nome ASC; /*lista todos os alunos ativos com seus dados
cadastros e telefones */
```

```
SELECT pk_mensalidade,
       fk_contrato,
       valor,
       data_vencimento,status
FROM tb_mensalidades
WHERE status = 'pago'
ORDER BY data_vencimento ASC; /*verifica as mensalidades pagas por ordem crescente
pela data de vencimento*/
```

/*Selects intermediarios*/

```
SELECT primeiro_nome, sobrenome, email
FROM tb_professores
WHERE pk_professor = (
SELECT fk_professor
FROM tb_folha_pagamento
ORDER BY salario_liquido DESC LIMIT 1 ); /*exibe o professor que possui o maior
salário líquido registrado na folha de pagamento*/
```

```
SELECT c.pk_contrato,
( SELECT COUNT(*) FROM tb_mensalidades m
WHERE m.fk_contrato = c.pk_contrato AND m.status = 'atrasado' ) AS total_atrasadas
FROM tb_contratos c; /*mostra a quantidade de mensalidades atrasadas vinculadas a
cada contrato*/
```

/*Selects avançados*/

```
SELECT c.pk_contrato, al.primeiro_nome, m.data_vencimento, p.valor_pago,
p.data_pagamento
FROM tb_contratos c
JOIN tb_alunos al ON c.fk_aluno = al.pk_aluno
JOIN tb_mensalidades m ON m.fk_contrato = c.pk_contrato
JOIN tb_pagamentos p ON p.fk_mensalidade = m.pk_mensalidade
WHERE p.data_pagamento = ( SELECT MAX(p2.data_pagamento)
FROM tb_pagamentos p2
JOIN tb_mensalidades m2 ON p2.fk_mensalidade = m2.pk_mensalidade
WHERE m2.fk_contrato = c.pk_contrato ); /*motra o pagamento mais recente realizado
para cada contrato, incluindo dados do aluno e da mensalidade*/
```

```
SELECT a.primeiro_nome, a.sobrenome, COUNT(m.fk_turma) AS total_matriculas
FROM tb_alunos a
JOIN tb_matriculas m
ON a.pk_aluno = m.fk_aluno
GROUP BY a.pk_aluno, a.primeiro_nome, a.sobrenome
HAVING COUNT(m.fk_turma) > (
SELECT AVG(qtd)
FROM (
SELECT COUNT(*) AS qtd
FROM tb_matriculas GROUP BY fk_aluno ) AS media_matriculas ); /*busca os alunos
que possuem mais matrículas do que a média geral de matrículas por aluno*/
```


/*Modelagem Estrela*/

```
CREATE DATABASE escola_idiomas_dw;
USE escola_idiomas_dw;

/*Dimensões*/
CREATE TABLE dim_aluno (
  sk_aluno INT PRIMARY KEY AUTO_INCREMENT,
  nk_aluno INT NOT NULL, /*ID original da tb_alunos*/
  nome_completo VARCHAR(160) NOT NULL,
  faixa_etaria VARCHAR(20) DEFAULT 'Não Informada',
  endereco VARCHAR(150) DEFAULT 'Não Informado',
  email VARCHAR(200) NOT NULL,
  raca VARCHAR(20) DEFAULT 'Não Informada');

CREATE TABLE dim_turma (
  sk_turma INT PRIMARY KEY AUTO_INCREMENT,
  nk_turma INT NOT NULL,
  nome_turma VARCHAR(50) NOT NULL,
  horario VARCHAR(20) NOT NULL,
  nome_nivel VARCHAR(20) NOT NULL,
  ordem_nivel INT NOT NULL,
  nome_idioma VARCHAR(20) NOT NULL );

CREATE TABLE dim_tempo (
  sk_tempo INT PRIMARY KEY, /*Formato numérico AAAAMMDD (ex: 20260509)*/
  data_completa DATE NOT NULL,
  dia INT NOT NULL,
  mes INT NOT NULL,
  nome_mes VARCHAR(20) NOT NULL,
  trimestre INT NOT NULL,
  semestre INT NOT NULL, ano INT NOT NULL,
  dia_semana VARCHAR(20) NOT NULL,
  fim_de_semana BOOLEAN NOT NULL );

CREATE TABLE dim_status_matricula ( sk_status_matricula INT PRIMARY KEY
  AUTO_INCREMENT, status_matricula VARCHAR(20) NOT NULL UNIQUE );

/*Tabela Fato*/
CREATE TABLE fato_matriculas (
  sk_fato_matricula INT PRIMARY KEY AUTO_INCREMENT,
  sk_aluno INT NOT NULL,
  sk_turma INT NOT NULL,
  sk_tempo INT NOT NULL,
  sk_status_matricula INT NOT NULL,

  /*Fatos para agregação analítica*/
  quantidade_matriculas INT DEFAULT 1,
  valor_matricula DECIMAL(10,2) DEFAULT 0.00,
```

```
desconto DECIMAL(10,2) DEFAULT 0.00,  
valor_final DECIMAL(10,2) DEFAULT 0.00,
```

```
FOREIGN KEY (sk_aluno) REFERENCES dim_aluno(sk_aluno),  
FOREIGN KEY (sk_turma) REFERENCES dim_turma(sk_turma),  
FOREIGN KEY (sk_tempo) REFERENCES dim_tempo(sk_tempo),  
FOREIGN KEY (sk_status_matricula) REFERENCES  
dim_status_matricula(sk_status_matricula));
```

/*Processo ETL*/

```
DELIMITER $$  
CREATE PROCEDURE sp_executar_etl()  
BEGIN  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;  
        RESIGNAL;  
    END;  
    START TRANSACTION;  
    INSERT INTO escola_idiomas_dw.dim_status_matricula (status_matricula)  
    SELECT DISTINCT status_curso  
    FROM escola_idiomas.tb_matriculas  
    WHERE status_curso NOT IN (SELECT status_matricula FROM  
escola_idiomas_dw.dim_status_matricula);  
  
    /*Atualiza ou insere novos alunos (Upsert incremental simples)*/  
    INSERT INTO escola_idiomas_dw.dim_aluno (nk_aluno, nome_completo, email)  
    SELECT  
        pk_aluno AS nk_aluno,  
        CONCAT(primeiro_nome, ' ', sobrenome) AS nome_completo,  
        email  
    FROM escola_idiomas.tb_alunos  
    ON DUPLICATE KEY UPDATE  
        nome_completo = VALUES(nome_completo),  
        email = VALUES(email);  
  
    /*CARGA DA DIM_TURMA (Extração com Junção/Desnormalização)*/  
    INSERT INTO escola_idiomas_dw.dim_turma (nk_turma, nome_turma, horario,  
nome_nivel, ordem_nivel, nome_idioma)  
    SELECT  
        t.pk_turma AS nk_turma,  
        t.nome_turma,  
        t.horario,  
        n.nome AS nome_nivel,  
        n.ordem AS ordem_nivel,  
        i.nome AS nome_idioma  
    FROM escola_idiomas.tb_turmas t  
    INNER JOIN escola_idiomas.tb_niveis n ON t.fk_nivel = n.pk_nivel
```

```

INNER JOIN escola_idiomas.tb_idiomas i ON n.fk_idioma = i.pk_idioma
ON DUPLICATE KEY UPDATE
    nome_turma = VALUES(nome_turma),
    horario = VALUES(horario),
    nome_nivel = VALUES(nome_nivel),
    ordem_nivel = VALUES(ordem_nivel),
    nome_idioma = VALUES(nome_idioma);

/*CARGA DA DIM_TEMPO (Geração dinâmica baseada nas matrículas)
*/
/*Garante que todas as datas de matrícula existentes no OLTP estejam cadastradas
na dim_tempo*/
INSERT IGNORE INTO escola_idiomas_dw.dim_tempo (sk_tempo, data_completa, dia,
mes, nome_mes, trimestre, semestre, ano, dia_semana, fim_de_semana)
SELECT DISTINCT
    CAST(DATE_FORMAT(data_matricula, '%Y%m%d') AS UNSIGNED) AS sk_tempo,
    data_matricula AS data_completa,
    DAY(data_matricula) AS dia,
    MONTH(data_matricula) AS mes,
    CASE MONTH(data_matricula)
        WHEN 1 THEN 'Janeiro' WHEN 2 THEN 'Fevereiro' WHEN 3 THEN 'Março'
        WHEN 4 THEN 'Abril'   WHEN 5 THEN 'Maio'       WHEN 6 THEN 'Junho'
        WHEN 7 THEN 'Julho'   WHEN 8 THEN 'Agosto'     WHEN 9 THEN 'Setembro'
        WHEN 10 THEN 'Outubro' WHEN 11 THEN 'Novembro' WHEN 12 THEN 'Dezembro'
    END AS nome_mes,
    QUARTER(data_matricula) AS trimestre,
    IF(MONTH(data_matricula) <= 6, 1, 2) AS semestre,
    YEAR(data_matricula) AS ano,
    CASE WEEKDAY(data_matricula)
        WHEN 0 THEN 'Segunda-feira' WHEN 1 THEN 'Terça-feira' WHEN 2 THEN
'Quarta-feira'
        WHEN 3 THEN 'Quinta-feira'  WHEN 4 THEN 'Sexta-feira' WHEN 5 THEN
'Sábado'
        WHEN 6 THEN 'Domingo'
    END AS dia_semana,
    IF(WEEKDAY(data_matricula) IN (5, 6), TRUE, FALSE) AS fim_de_semana
FROM escola_idiomas.tb_matriculas;

/*CARGA DA TABELA FATO_MATRICULAS (Lookup de SKs e Carga)*/
TRUNCATE TABLE escola_idiomas_dw.fato_matriculas;
INSERT INTO escola_idiomas_dw.fato_matriculas (
    sk_aluno,
    sk_turma,
    sk_tempo,
    sk_status_matricula,
    quantidade_matriculas,
    valor_matricula,
    desconto,
    valor_final
)

```

```

SELECT
    da.sk_aluno,
    dt.sk_turma,
    CAST(DATE_FORMAT(m.data_matricula, '%Y%m%d') AS UNSIGNED) AS sk_tempo,
    ds.sk_status_matricula,
    1 AS quantidade_matriculas,
    /*Buscando valores financeiros fictícios/padrão ou associados*/
    COALESCE(men.valor, 0.00) AS valor_matricula,
    0.00 AS desconto,
    COALESCE(men.valor, 0.00) AS valor_final
FROM escola_idiomas.tb_matriculas m
INNER JOIN escola_idiomas_dw.dim_aluno da ON m.fk_aluno = da.nk_aluno
INNER JOIN escola_idiomas_dw.dim_turma dt ON m.fk_turma = dt.nk_turma
INNER JOIN escola_idiomas_dw.dim_status_matricula ds ON m.status_curso =
ds.status_matricula
LEFT JOIN escola_idiomas.tb_contratos c ON m.fk_aluno = c.fk_aluno
LEFT JOIN (
    SELECT fk_contrato, MIN(valor) AS valor
    FROM escola_idiomas.tb_mensalidades
    GROUP BY fk_contrato
) men ON c.pk_contrato = men.fk_contrato;

COMMIT;
END$$

DELIMITER ;

```

/*Validação OLTP e OLAP*/

```
CALL sp_executar_etl();
```

```
SELECT
status_curso AS status,
COUNT(*) AS total_matriculas
FROM escola_idiomas.tb_matriculas
GROUP BY status_curso;                                /*consulta oltp*/
```

```
SELECT ds.status_matricula AS status,
SUM(f.quantidade_matriculas) AS total_matriculas
FROM escola_idiomas_dw.fato_matriculas f
INNER JOIN escola_idiomas_dw.dim_status_matricula ds ON f.sk_status_matricula =
ds.sk_status_matricula GROUP BY ds.status_matricula;    /*consulta olap/
```

```
SELECT i.nome AS idioma, SUM(men.valor) AS receita_total
FROM escola_idiomas.tb_matriculas m
INNER JOIN escola_idiomas.tb_turmas t ON m.fk_turma = t.pk_turma
INNER JOIN escola_idiomas.tb_niveis n ON t.fk_nivel = n.pk_nivel
INNER JOIN escola_idiomas.tb_idiomas i ON n.fk_idioma = i.pk_idioma
LEFT JOIN escola_idiomas.tb_contratos c ON m.fk_aluno = c.fk_aluno
LEFT JOIN (
SELECT fk_contrato, MIN(valor) AS valor
FROM escola_idiomas.tb_mensalidades
GROUP BY fk_contrato ) men ON c.pk_contrato = men.fk_contrato GROUP BY i.nome;
/*consulta oltp*/
```

```
SELECT
dt.nome_idioma AS idioma,
SUM(f.valor_matricula) AS receita_total
FROM escola_idiomas_dw.fato_matriculas f
INNER JOIN escola_idiomas_dw.dim_turma dt ON f.sk_turma = dt.sk_turma
GROUP BY dt.nome_idioma; /*consulta olap/
```

```

1 • SELECT
2
3     status_curso AS status,
4
5     COUNT(*) AS total_matriculas
6
7 FROM escola_idiomas.tb_matriculas
8
9 GROUP BY status_curso; /*consulta oltp*/

```

Result Grid  Filter Rows: Export:  Wrap Cell Content: 

	status	total_matriculas
▶	ativo	4
	trancado	1

```

1 • SELECT ds.status_matricula AS status,
2
3     SUM(f.quantidade_matriculas) AS total_matriculas
4
5 FROM escola_idiomas_dw.fato_matriculas f
6
7 INNER JOIN escola_idiomas_dw.dim_status_matricula ds ON f.sk_status_matricula = ds.sk_status_matricula GROUP BY ds.status_matricula;

```

Result Grid  Filter Rows: Export:  Wrap Cell Content: 

	status	total_matriculas
▶	ativo	4
	trancado	1

Limit to 1000 rows

```

1 • SELECT i.nome AS idioma, SUM(men.valor) AS receita_total
2 FROM escola_idiomas.tb_matriculas m
3 INNER JOIN escola_idiomas.tb_turmas t ON m.fk_turma = t.pk_turma
4 INNER JOIN escola_idiomas.tb_niveis n ON t.fk_nivel = n.pk_nivel
5 INNER JOIN escola_idiomas.tb_idiomas i ON n.fk_idioma = i.pk_idioma
6 LEFT JOIN escola_idiomas.tb_contratos c ON m.fk_aluno = c.fk_aluno
7 LEFT JOIN (
8 SELECT fk_contrato, MIN(valor) AS valor
9 FROM escola_idiomas.tb_mensalidades
10 GROUP BY fk_contrato ) men ON c.pk_contrato = men.fk_contrato GROUP BY i.nome; /*consulta oltp*/

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

	idioma	receita_total
▶	Espanhol	320.00
	Francês	380.00
	Inglês	1050.00

Limit to 1000 rows

```

1 • SELECT
2
3     dt.nome_idioma AS idioma,
4
5     SUM(f.valor_matricula) AS receita_total
6
7 FROM escola_idiomas_dw.fato_matriculas f
8
9 INNER JOIN escola_idiomas_dw.dim_turma dt ON f.sk_turma = dt.sk_turma
10
11 GROUP BY dt.nome_idioma; /*consulta olap/

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

	idioma	receita_total
▶	Espanhol	320.00
	Francês	380.00
	Inglês	1050.00

Uma Análise Para o futuro: melhoria da estrutura para BI e IA

Como apresentado no início do projeto, a implantação do ERP SisGESC não se limita apenas à organização das atividades da instituição de ensino. O sistema também abre possibilidades para futuras integrações com tecnologias avançadas de análise de dados.

Nesse contexto, a adoção de ferramentas de **Business Intelligence (BI)** e Inteligência Artificial surge como uma evolução natural, permitindo apoiar a tomada de decisões estratégicas com base em dados.

A utilização de BI possibilita a análise de dados históricos e atuais por meio de dashboards e relatórios, auxiliando na identificação de padrões e indicadores importantes.

A seguir, são apresentadas propostas de melhorias e expansões com foco em BI e IA.

- Inclusão do campo **origem_captacao** na tabela de alunos, permitindo avaliar a efetividade de estratégias de marketing
- Inclusão do campo **motivo_cancelamento** na tabela de contratos, facilitando a análise das causas de evasão

Já no Uso da Inteligência Artificial, o sistema pode evoluir para prever comportamentos e apoiar ações preventivas como:

- **Previsão de evasão escolar**, com base em fatores como atrasos de pagamento, status da matrícula e histórico do aluno
- **Análise de inadimplência**, identificando padrões financeiros que podem levar ao abandono
- **Recomendação de cursos ou turmas**, com base no perfil e objetivo do aluno (**motivo_estudo**)

Além disso, propõe-se a criação de uma tabela de rendimento acadêmico, contendo dados como frequência e desempenho. Esses dados permitem identificar alunos com baixo engajamento, um dos principais indicativos de evasão.

Dessa forma, o uso integrado de BI e IA possibilita não apenas analisar o cenário atual, mas também agir de forma preventiva, reduzindo a evasão escolar e contribuindo para a melhoria da gestão e da qualidade de ensino;

Link git hub disponível em:

<https://github.com/LeoAraujo34/sisgesc-banco-dados>